El montículo mínimo (Min-heap): SplMinHeap

Caso práctico

Carlos prepara la web de una carrera popular. Los tiempos de los corredores llegan **desordenados** desde los dorsales electrónicos, y la pantalla de meta debe mostrar en todo momento **el mejor tiempo**. Ordenar todos los tiempos cada vez que llega uno sería un desperdicio. **María** le propone un **montículo mínimo**: el menor siempre está a mano.

1.- Qué es un montículo mínimo

Un **montículo** (*heap*) es un **árbol binario casi completo** (todos los niveles llenos salvo, quizá, el último, que se llena de izquierda a derecha) que cumple la **propiedad de montículo**. En un **montículo mínimo**, cada nodo es **menor o igual que sus hijos**, así que el **mínimo está siempre en la raíz**.

No está ordenado como una lista: entre hermanos o primos no hay ningún orden. Pero esa propiedad basta para:

Operación	Qué hace	En SplMinHeap	Coste
insert	Añadir un valor	`insert($x)`	O(log n)
extractMin	Sacar el mínimo	`extract()`	O(log n)
peek	Ver el mínimo	`top()`	O(1)
size, *isEmpty*	Tamaño / ¿vacío?	`count($h)`, `isEmpty()`	O(1)

Al insertar, el valor se pone al final y "sube" mientras sea menor que su padre; al extraer, el último pasa a la raíz y "baja" hasta su sitio. En los dos casos se recorre un solo camino del árbol, de altura log n.

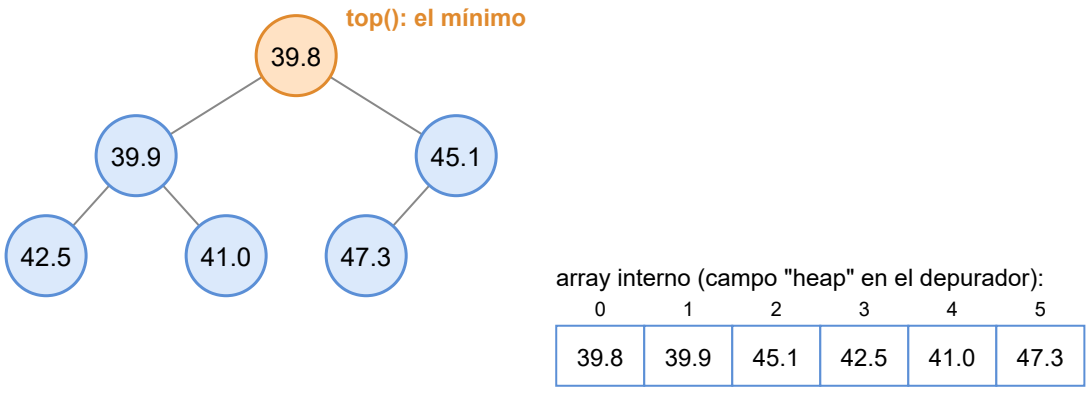


Figura 1. Un montículo mínimo y el array en que se guarda: los hijos de la posición i están en 2i+1 y 2i+2

2.- La clase SplMinHeap

`SplMinHeap` hereda de la clase abstracta `SplHeap` (carpeta `07-CustomHeap-SplHeap`) e implementa `Iterator` y `Countable`. Compara los valores con el operador `<=>`, así que sirve para números, textos, arrays y objetos comparables como `DateTime`.

```
<?php
$tiempos = new SplMinHeap();
foreach ([42.5, 39.8, 45.1, 41.0] as $t) {
    $tiempos->insert($t);
    echo "El mejor tiempo es ", $tiempos->top(), PHP_EOL;   // 42.5, 39.8, 39.8, 39.8
}
echo $tiempos->extract();       // 39.8
echo $tiempos->extract();       // 41.0
```

- **Textos**: se comparan byte a byte, así que las mayúsculas van antes que las minúsculas ("Uva" < "kiwi").
- **Arrays**: se comparan elemento a elemento. Insertar `[prioridad, dato]` permite ordenar por la prioridad y desempatar por el dato (o por un número de orden).
- **Fechas**: `DateTime` y `DateTimeImmutable` se comparan por el instante.
- Con el montículo vacío, `extract()` y `top()` lanzan `RuntimeException`.

Debes conocer

Recorrer un montículo lo vacía. `foreach` (y `iterator_to_array`) van **extrayendo** los elementos en orden, igual que en `SplPriorityQueue`. Para verlos sin perderlos, recorre una copia: `foreach (clone $heap as $x)`. Además, el campo `heap` que se ve en el depurador **no** está ordenado: solo cumple la propiedad de montículo.

3.- ¿SplMinHeap, SplPriorityQueue o sort()?

- Si solo necesitas **ordenar un array una vez**, usa `sort()`: es más simple y rápido.
- Si los datos **llegan poco a poco** y en cada momento necesitas el **menor**, usa `SplMinHeap`.
- Si cada elemento tiene una **prioridad distinta del propio valor** (una tarea y su urgencia), `SplPriorityQueue` es más cómoda; con un montículo, se guarda `[prioridad, valor]`.
- Si necesitas el **mayor**, usa `SplMaxHeap` (o inserta los números con el signo cambiado).

4.- Aplicaciones típicas

Problema	Cómo
El mínimo de un flujo de datos (mejor tiempo, precio más bajo, próxima cita)	`top()` siempre da el menor
Los K mayores de muchísimos datos	Un min-heap de tamaño K: su raíz es el peor de los K mejores
Fusionar K listas ordenadas (registros de varios servidores)	El montículo guarda el primero pendiente de cada lista
Asignar trabajos a la máquina o sala que antes queda libre	El montículo guarda la hora en que se libera cada una
Planificación de eventos por fecha, algoritmos de grafos (Dijkstra, Prim)	Siempre se procesa el más próximo
Ordenar (*heapsort*)	Insertar todo y extraer todo: O(n log n)

5.- Ejercicio resuelto: los K mayores con un montículo mínimo

Ejercicio resuelto

Un juego online registra miles de partidas. Muestra el **top 3** de puntuaciones sin guardar ni ordenar todas.

```php
<?php
function topK(iterable $puntuaciones, int $k): array
{
    $mejores = new SplMinHeap();
    foreach ($puntuaciones as $p) {
        if (count($mejores) < $k) {
            $mejores->insert($p);
        } elseif ($p > $mejores->top()) {     // mejor que el PEOR de los K mejores
            $mejores->extract();
            $mejores->insert($p);
        }
    }
    return array_reverse(iterator_to_array($mejores, false));   // de mayor a menor
}
function partidas(): Generator
{
    mt_srand(2026);
    for ($i = 0; $i < 10_000; $i++) {
        yield mt_rand(0, 99_999);
    }
    yield 100_500;                            // un récord al final
```

```
}
print_r(topK(partidas(), 3));                // [100500, 99989, 99984]
```

¿Por qué un montículo **mínimo** para buscar los **mayores**? Porque lo que necesitamos consultar a cada paso es el **peor** de los K que llevamos: si la nueva puntuación lo supera, lo echamos. El montículo nunca tiene más de K elementos, así que la memoria es mínima aunque haya millones de partidas, y cada paso cuesta O(log K).

Autoevaluación

- En un montículo mínimo, ¿dónde está el menor elemento? **En la raíz: `top()`.**
- ¿Está ordenado el array interno de un montículo? **No: solo cumple que cada padre es menor o igual que sus hijos.**
- ¿Qué ocurre si recorres un `SplMinHeap` con `foreach`? **Se vacía: cada vuelta extrae el mínimo.**
- ¿Qué tipo de montículo usarías para mantener los 10 mayores de un flujo de datos? **Un montículo mínimo de tamaño 10.**

6.- Ejercicios

Fichero	Contenido
`Ejercicios MinHeap/e01-SplMinHeap.php`	La API: `insert`, `top`, `extract`, `foreach` (sobre una copia), textos, arrays y fechas, *heapsort*
`Ejercicios MinHeap/e02-SplMinHeap.php`	Aplicaciones: top K, fusionar registros de varios servidores, máquina que antes queda libre
`Ejercicios MinHeap/problemas/`	`e01` y `e02`: problemas y soluciones